

Data Structure Use Cases

Use Case 1: Customer transaction history

Scenario: Track all purchases made by a customer in order.

transactions = [250, 399, 120, 899]

Why list?

- Order matters (time sequence)
- Duplicate purchases allowed
- New transactions keep coming (mutable)

Real-world: E-commerce, banking, fintech

Use Case 2: ETL batch processing

Scenario: Store files to be processed in a pipeline.

files_to_process = ["jan.csv", "feb.csv", "mar.csv"]

Pipelines iterate sequentially → list is ideal.

Use Case 3: Database record representation

Scenario: Fetch row from database.

customer_record = (101, "Prachi", "Ahmedabad")

Why tuple?

- Data should not change accidentally
- Represents fixed schema
- Memory efficient

Real-world: DB connectors, Snowflake fetch results

Use Case 4: Coordinates / geospatial data

location = (23.0225, 72.5714)

Used in:

- delivery apps
- ride sharing
- store mapping

Coordinates are fixed → tuple preferred.

Use Case 5: Remove duplicate customers

customer_ids = [101, 102, 101, 103]

unique_customers = set(customer_ids)

Why set?

- Ensures uniqueness
- Very fast deduplication

Use Case 6: Fraud detection (common users)

```
website_users = {101, 102, 103}
```

```
app_users = {102, 103, 104}
```

```
common_users = website_users.intersection(app_users)
```

Used to find:

- overlapping users
- suspicious activity
- cross-platform behavior

Use Case 7: Permission systems

```
user_permissions = {"read", "write", "execute"}
```

Use Case 8: Customer master data

```
customer = {  
    "id": 101,  
    "name": "Prachi",  
    "city": "Ahmedabad"  
}
```

Why dict?

- Structured data
- Fast lookup by key
- JSON-like format

Use Case 9: Aggregation / counting

```
sales_by_region = {  
    "North": 120000,  
    "South": 95000  
}
```

Used heavily in:

- KPI calculations
- reporting
- dashboards